

Data Platform Engineer — Home Assignment

Estimated time: 3–4 hours

Background

You are joining a data engineering team that ingests sensor telemetry from a fleet of ships into a centralized analytics platform. Ships periodically generate JSON files containing sensor measurements (GPS) and upload them to storage. Your job is to build the pipeline that ingests that data into a queryable data store.

You are free to choose your own tools. What matters is how you design and orchestrate the pipeline.

What you need to do

Create an ingestion pipeline to ingest JSON files containing sensor data and transform them into a columnar format.

You decide:

- The number of stages in the pipeline and the boundaries between each stage
- The message queue or triggering mechanism that should be used
- The approach for handling chunking, batching, and file sizing for the selected data store
- The components that the data pipeline should include to make it as production-ready as possible within the given time constraints, prioritizing the most critical real-world functionality and reliability requirements

The most critical aspect is designing for parallel processing and large scale, the pipeline must handle many concurrent ships, variable file sizes, and bursty loads while guaranteeing that no data is lost

Requirements

	Detail
Latency	Up to 10–15 minutes
Scale	The solution should be scalable, supporting multiple ship dumps and enabling parallel processing.
Parallel processing	Multiple files from different ships must be processed concurrently.
Volume	File sizes vary significantly and are unpredictable, need to support small and large file sizes. Files are not evenly distributed across time or sources.
Data completeness	The target DB should include all the data from the source
Resource efficiency	The pipeline should be efficient
Error handling	The pipeline needs to handle failures at every stage without data loss
Schema validation	Should be able to handle schema changes

Input Data

Each ship dumps every period of time a JSON file that contains an array of GPS sensor records:

JSON

```
[
  {
    "source_id": "ship-42",
    "measurement": "gps",
    "timestamp": "2026-05-23T10:14:00Z",
    "values": {
      "lat": 32.08,
      "lon": 34.78,
      "speed_knots": 12.4
    }
  },
]
```

```
...  
]
```

File Naming

{source_id}_{measurement}_{timestamp_utc}.json

Example: ship-42_gps_20260523T101400Z.json

Available Measurement

Measurement	Fields
gps	lat, lon, speed_knots

Generator

The **generator.py** code is for your assistance in simulating the JSON files produced by ships dumping sensor telemetry data into storage. The generator is provided as a helper utility to produce periodic ship telemetry dumps. **You can use it as-is or implement your own generator if preferred.**

Generator parameters:

Env var	Default	Description
OUTPUT-DIR	-	Where to store the sensor JSON files
RATE	2	files per minute
SIZE	small	small (~15 KB), medium (~5 MB), large (~45 MB)

SOURCES	10	number of distinct ship IDs
---------	----	-----------------------------

Running the generator:

Python

Prerequisites: Python 3.10+, no extra packages required

Small files (~15 KB), 2 files/min, 10 ships

```
python generator.py --output-dir ./input --rate 2 --size small
```

Medium files (~5 MB), 1 file/min, 20 ships

```
python generator.py --output-dir ./input --rate 1 --size medium
--sources 20
```

Large files (~45 MB), slow rate – for testing chunking logic

```
python generator.py --output-dir ./input --rate 0.5 --size large
```

All flags

```
python generator.py \
  --output-dir ./input \    # directory to drop files into
  --rate 2 \                # files per minute (default: 2)
  --size small \           # small | medium | large (default: small)
  --sources 10             # number of distinct ship IDs (default:
10)
```

More instructions

- Use **free and open-source** tools - no paid cloud services
- **Treat this as a production workflow.** The pipeline must be robust and stable. It is expected to think about scenarios that could break the pipeline and handle them in the code. Document the scenarios you chose to address in the README and justify your choices.
- The output should be stored in a queryable format

- Use `generator.py` (or build your own) to simulate the JSON files produced by ships dumping sensor telemetry data into storage.
-

What You Need To Deliver

1. Working Pipeline

A Git repository containing all code.

Include a `README.md` with setup instructions. Someone unfamiliar with your choices should be able to run the pipeline by following the README.

2. Design Document (1–2 pages)

A written document covering:

- Your pipeline architecture (stages, boundaries, data flow)
- Why did you choose each tool (queue, storage, data store)
- How you handle failures — what happens if a stage crashes mid-processing?
- Which components did you choose to focus on in the data pipeline, and why
- Trade-offs you made given the 3–4 hour time constraint
- What would you change for a production deployment at 500+ sources

3. AI Transcript

The complete conversation log from Claude Code, Cursor, Copilot, or any AI coding tool you used during the assignment.

We expect candidates to use AI tools. We want to see **how** you work with them. Export the full session.

Submission

Send:

1. A link to your Git repository (GitHub, GitLab, or a zip file)
2. The design document (can be in the repo or a separate PDF)
3. The AI transcript file(s)

Good luck.